

Fixed-Size Synaptic Memory vs. the Growing KV Cache

Concept: Associative Memory and Fast Weights | DataForge 2026, Pathway Track

The design pressure

A standard Transformer answers a question about something said earlier by keeping every past token's key and value vector in a KV cache and comparing the current query against all of them. This is exact and simple, but the cache grows linearly with context length: doubling the conversation doubles the memory. Fast-weight and linear-attention research reframes this differently: instead of storing tokens, compress the past into a fixed-size state and update that state as an online learning rule (Sun et al., 2024, arXiv:2407.04620). The central open question this framing raises is not whether a fixed state can work — it clearly can, at some scale — but how its accuracy degrades as more information is written into a container that never grows.

What changes technically, and the trade-off

This project implements the simplest possible version of that idea: a dense Hebbian fast-weight matrix W ($\text{dim} \times \text{dim}$), updated as $W_t = \lambda \cdot W_{(t-1)} + \eta \cdot (v k^T)$ and read as $W \cdot \text{query}$. Unlike the KV cache, W 's size never changes with sequence length: it is $O(1)$ in space for a fixed dimension, versus the KV cache's $O(N)$. The cost is interference — new outer-product writes are superimposed on old ones in the same matrix, so old associations can be overwritten or blurred rather than evicted cleanly. On a controlled associative-recall benchmark built for this project (random key/value vectors, no semantic redundancy for either system to exploit), synaptic-memory retrieval accuracy on a fixed set of 20 tracked facts falls from 100% at 50 total facts to roughly 5–15% by 1,000–10,000 facts, while the KV cache stays at or near 100% throughout — at the cost of its memory growing roughly 300× over the same range. These are this project's own measurements (seed-fixed, reproducible via experiments/benchmark.py), not an independent evaluation.

Where this sits among recent systems

System	State w.r.t. length	Handles interference via	Evidence type
KV cache / full attention	Grows, $O(N)$	N/A (exact storage)	Well-established baseline
Linear attention w/ fixed state (Arora et al., 2024, arXiv:2402.18668)	Fixed, $O(1)$	Better feature maps / hybrid recall-throughput tuning	Benchmarked (public)
Dragon Hatchling (BDH) (Kosowski et al., 2025, arXiv:2509.26507)	Fixed, $O(1)$ (neuron-graph state)	Sparse, non-negative, scale-free connectivity	Benchmarked; reported long-context results
This prototype	Fixed, $O(1)$	None (dense, unconstrained)	Own controlled benchmark

The comparison is not that one system is strictly better: linear-attention variants trade some recall for throughput and are competitive when tuned (Arora et al., 2024); BDH's sparsity is specifically motivated by reducing the interference this project measures in its plainest, unconstrained form (Kosowski et al., 2025). A companion line of recent work studies the same forgetting problem directly and proposes an exact auxiliary memory to fix it (Fang et al., 2025, arXiv:2607.02303) — this project's degradation curve is best read as an illustration of the problem that motivates work like theirs, not a critique of any specific system.

Role of BDH and BDH-CQ

BDH's relevance is direct: it reformulates attention as Hebbian synaptic memory updated as the model reads, which is the same update family this prototype isolates, minus BDH's sparsity and scale-free connectivity constraints (Kosowski et al., 2025). BDH-CQ's relevance is more specific: it stores task demonstrations in recurrent memory at inference time and reasons iteratively in a continuous latent workspace rather than a verbalized chain-of-thought, reaching 29.5% pass@2 on the public ARC-AGI-1 evaluation set at roughly \$0.0007 per task — a result independently reproduced by a team from Bielik AI and NYU, which is a genuine independent evaluation, distinct from a benchmark run by the system's own developers. BDH-CQ's contribution to this project is conceptual (adaptation living in state, not weights) rather than a component this prototype reuses.

Most important limitation

This prototype's dense, unconstrained Hebbian update is a deliberately weak baseline: it has none of the structural features (sparsity, non-negativity, monosemantic synapses) that BDH's own paper connects to lower interference. Its degradation curve should be read as an upper bound on the problem fixed-size memory faces in the worst case, not as a measurement of how BDH itself would perform on the same task — that comparison remains untested here and is the natural next experiment.

Sources: Kosowski et al. 2025 (arXiv:2509.26507); Pathway 2026, BDH-CQ (arXiv:2608.09888); Arora et al. 2024 (arXiv:2402.18668); Sun et al. 2024 (arXiv:2407.04620); Fang et al. 2025 (arXiv:2607.02303). Full citations and reproduction instructions in README.md.